

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

DATE	28-02-2026
TEAM ID	LTVIP2026TMIDS89121
PROJECT NAME	Exploratory Analysis of Rain Fall Data in India for Agriculture
MAXIMUM MARKS	3MARKS

DATA PRE-PROCESSING:

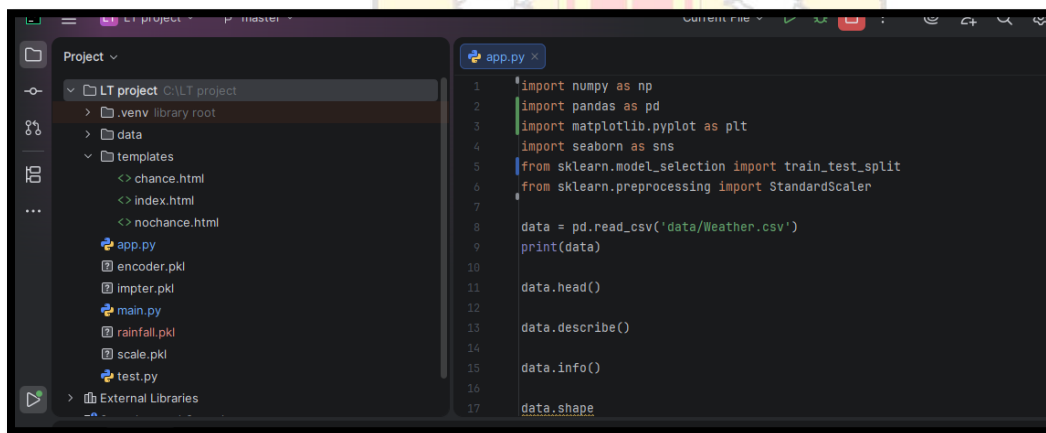
Data Pre-processing is one of the most important steps in any Machine Learning project. Raw data cannot be directly used for analysis or model building. It must be cleaned, organized, and transformed into a structured format.

Data Pre-processing in this project includes the following main tasks:

1.Importing the Libraries:

Before working with the dataset, necessary Python libraries must be imported. These libraries provide functions required for data handling, visualization, and model building.

- import numpy as np
- import pandas as pd
- import matplotlib.pyplot as plt
- import seaborn as sns
- from sklearn.model_selection import train_test_split
- from sklearn.preprocessing import StandardScaler

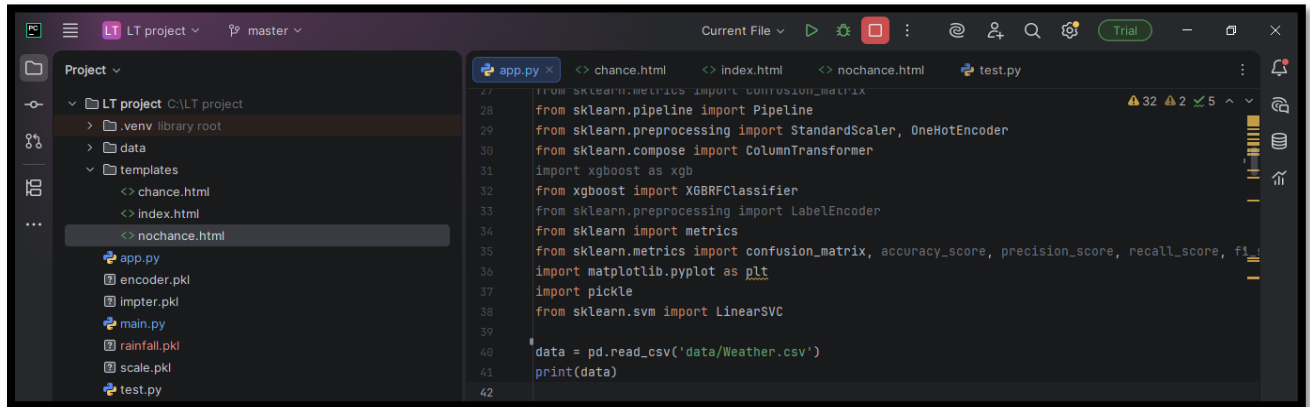


```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.model_selection import train_test_split
6 from sklearn.preprocessing import StandardScaler
7
8 data = pd.read_csv('data/Weather.csv')
9 print(data)
10
11 data.head()
12
13 data.describe()
14
15 data.info()
16
17 data.shape
```

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

IMPORTING THE DATASET:

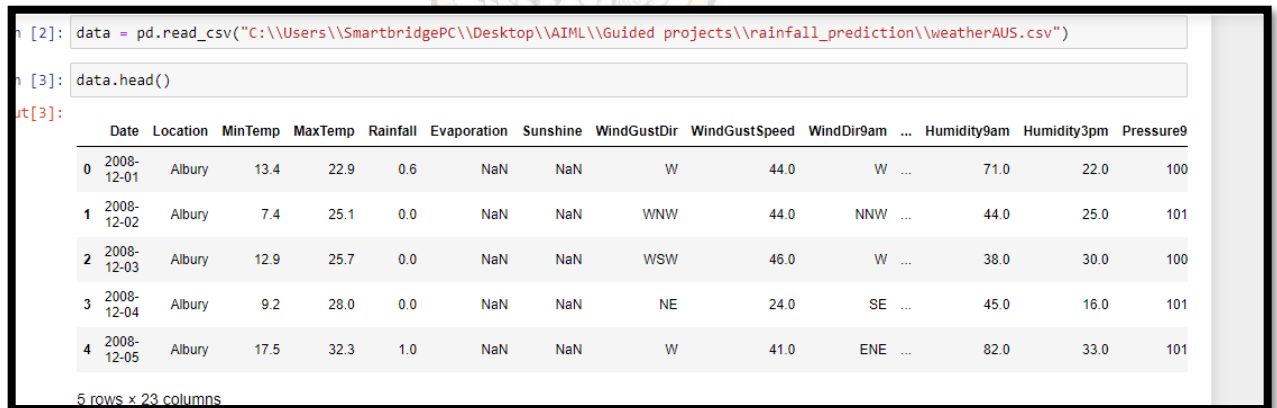
- Now we have to import the csv file on app.py
- Just like below image



```
27 from sklearn.metrics import confusion_matrix
28 from sklearn.pipeline import Pipeline
29 from sklearn.preprocessing import StandardScaler, OneHotEncoder
30 from sklearn.compose import ColumnTransformer
31 import xgboost as xgb
32 from xgboost import XGBRFClassifier
33 from sklearn.preprocessing import LabelEncoder
34 from sklearn import metrics
35 from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score
36 import matplotlib.pyplot as plt
37 import pickle
38 from sklearn.svm import LinearSVC
39
40 data = pd.read_csv('data/Weather.csv')
41 print(data)
42
```

ANALYSE THE DATA:

- head () method is used to return top n (5 by default) rows of a Data Frame or



```
[2]: data = pd.read_csv("C:\\Users\\SmartbridgePC\\Desktop\\AIIML\\Guided projects\\rainfall_prediction\\weatherAUS.csv")
[3]: data.head()
```

	Date	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindGustDir	WindGustSpeed	WindDir9am	...	Humidity9am	Humidity3pm	Pressure9
0	2008-12-01	Albury	13.4	22.9	0.6	NaN	NaN	W	44.0	W	...	71.0	22.0	100
1	2008-12-02	Albury	7.4	25.1	0.0	NaN	NaN	WNW	44.0	NNW	...	44.0	25.0	101
2	2008-12-03	Albury	12.9	25.7	0.0	NaN	NaN	WSW	46.0	W	...	38.0	30.0	100
3	2008-12-04	Albury	9.2	28.0	0.0	NaN	NaN	NE	24.0	SE	...	45.0	16.0	101
4	2008-12-05	Albury	17.5	32.3	1.0	NaN	NaN	W	41.0	ENE	...	82.0	33.0	101

5 rows x 23 columns

series.

- Describe () method computes a summary of statistics like count, mean, standard deviation, min, max and quartile values.

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

The output is as shown below

```
In [ ]: data.describe() # gives the descriptive statistics of the data
```

	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindGustSpeed	Wind Speed9am	Wind Speed3pm	Humidity9am	Humidity3pm
count	143975.000000	144199.000000	142199.000000	82670.000000	75625.000000	135197.000000	143693.000000	142398.000000	142806.000000	140953.000000
mean	12.194034	23.221348	2.360918	5.468232	7.611178	40.035230	14.043426	18.662657	68.880831	51.5391
std	6.398495	7.119049	8.478060	4.193704	3.785483	13.607062	8.915375	8.809800	19.029164	20.7959
min	-8.500000	-4.800000	0.000000	0.000000	0.000000	6.000000	0.000000	0.000000	0.000000	0.000000
25%	7.600000	17.900000	0.000000	2.600000	4.800000	31.000000	7.000000	13.000000	57.000000	37.000000
50%	12.000000	22.600000	0.000000	4.800000	8.400000	39.000000	13.000000	19.000000	70.000000	52.000000
75%	16.900000	28.200000	0.800000	7.400000	10.600000	48.000000	19.000000	24.000000	83.000000	66.000000
max	33.900000	48.100000	371.000000	145.000000	14.500000	135.000000	130.000000	87.000000	100.000000	100.000000

- From the data we infer that there are only decimal values and no categorical values info () gives information about the data.

```
In [ ]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 145460 entries, 0 to 145459
Data columns (total 23 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Date                  145460 non-null object
1   Location              145460 non-null object
2   MinTemp              143975 non-null float64
3   MaxTemp              144199 non-null float64
4   Rainfall             142199 non-null float64
5   Evaporation          82670 non-null float64
6   Sunshine             75625 non-null float64
7   WindGustDir          135134 non-null object
8   WindGustSpeed        135197 non-null float64
9   WindDir9am           134894 non-null object
10  WindDir3pm           141232 non-null object
11  WindSpeed9am         143693 non-null float64
12  WindSpeed3pm         142398 non-null float64
13  Humidity9am          142806 non-null float64
14  Humidity3pm          140953 non-null float64
15  Pressure9am          130395 non-null float64
16  Pressure3pm          130432 non-null float64
17  Cloud9am             89572 non-null float64
18  Cloud3pm             86102 non-null float64
19  Temp9am              143693 non-null float64
20  Temp3pm              141851 non-null float64
```

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

- data.shape: give the information about the dimension of the dataset, such as no. of rows and no. of columns

```
data.shape # gives the dimensions of the data  
(145460, 19)
```

Handling Missing Values:

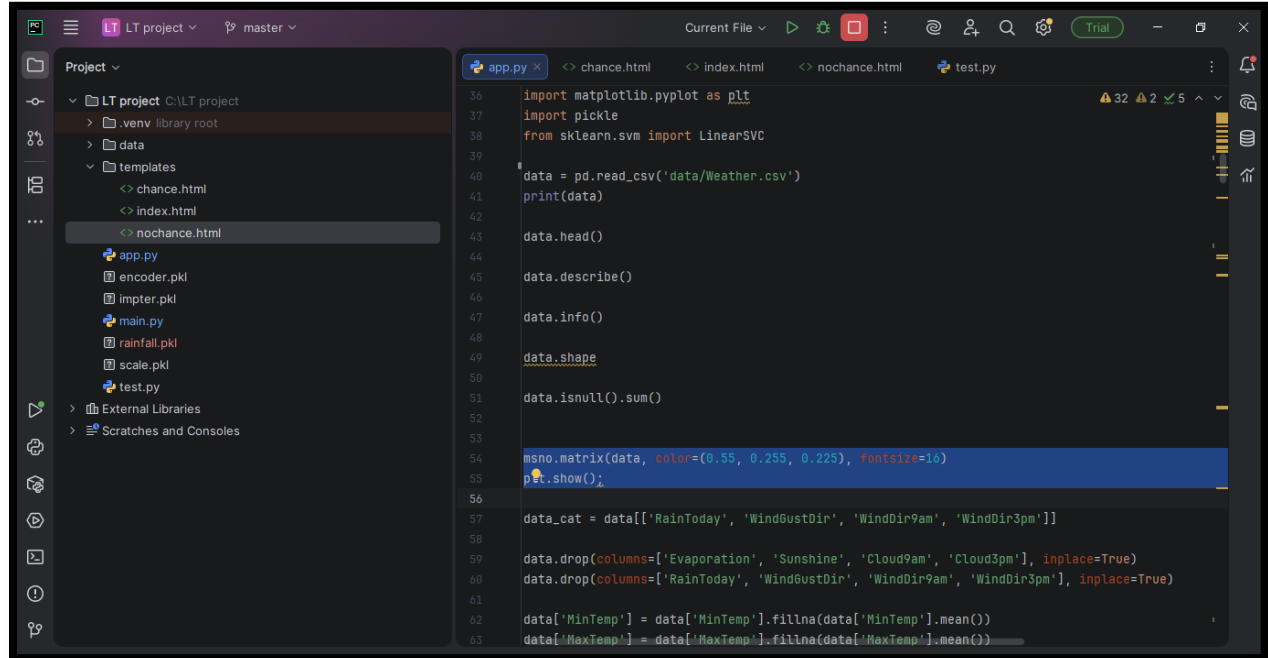
1. After loading it is important to check the complete information of data as it can indicate many of the hidden information such as null values in a column or a row
2. Check whether any null values are there or not. if it is present then the following can be done,

```
5]: data.isnull().sum()  
5]: Date          0  
   Location        0  
   MinTemp        1485  
   MaxTemp        1261  
   Rainfall       3261  
   Evaporation    62790  
   Sunshine       69835  
   WindGustDir    10326  
   WindGustSpeed  10263  
   WindDir9am     10566  
   WindDir3pm     4228  
   WindSpeed9am   1767  
   WindSpeed3pm   3062  
   Humidity9am    2654  
   Humidity3pm    4507  
   Pressure9am    15065  
   Pressure3pm    15028  
   Cloud9am       55888  
   Cloud3pm       59358  
   Temp9am        1767  
   Temp3pm        3609  
   RainToday      3261  
   RainTomorrow   3267  
   dtype: int64
```

3. **Missing matrix:** It is a way of representing the data in the 2-D form. It gives a colored visual summary of the data, missing no matrix is used to identify the missing data in the dataset, which is represented as white color gaps

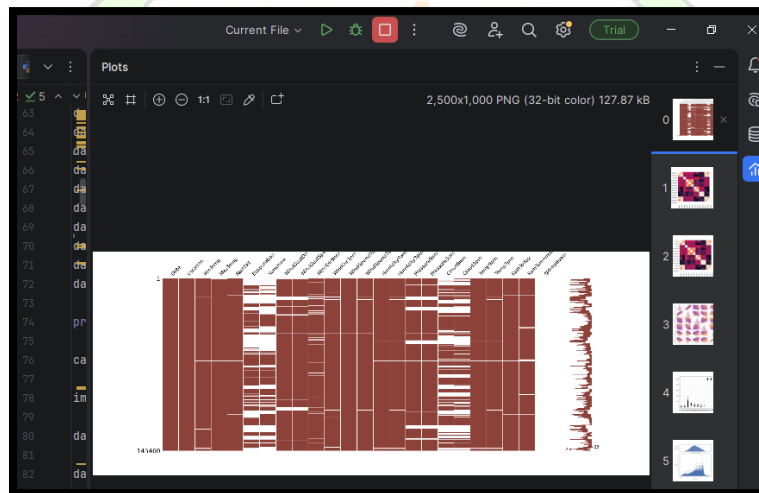
EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

We have to paste the code



```
36 import matplotlib.pyplot as plt
37 import pickle
38 from sklearn.svm import LinearSVC
39
40 data = pd.read_csv('data/Weather.csv')
41 print(data)
42
43 data.head()
44
45 data.describe()
46
47 data.info()
48
49 data.shape
50
51 data.isnull().sum()
52
53
54 sns.matrix(data, color=(0.55, 0.255, 0.225), fontsize=10)
55 plt.show()
56
57 data_cat = data[['RainToday', 'WindGustDir', 'WindDir9am', 'WindDir3pm']]
58
59 data.drop(columns=['Evaporation', 'Sunshine', 'Cloud9am', 'Cloud3pm'], inplace=True)
60 data.drop(columns=['RainToday', 'WindGustDir', 'WindDir9am', 'WindDir3pm'], inplace=True)
61
62 data['MinTemp'] = data['MinTemp'].fillna(data['MinTemp'].mean())
63 data['MaxTemp'] = data['MaxTemp'].fillna(data['MaxTemp'].mean())
```

The output should be like this:



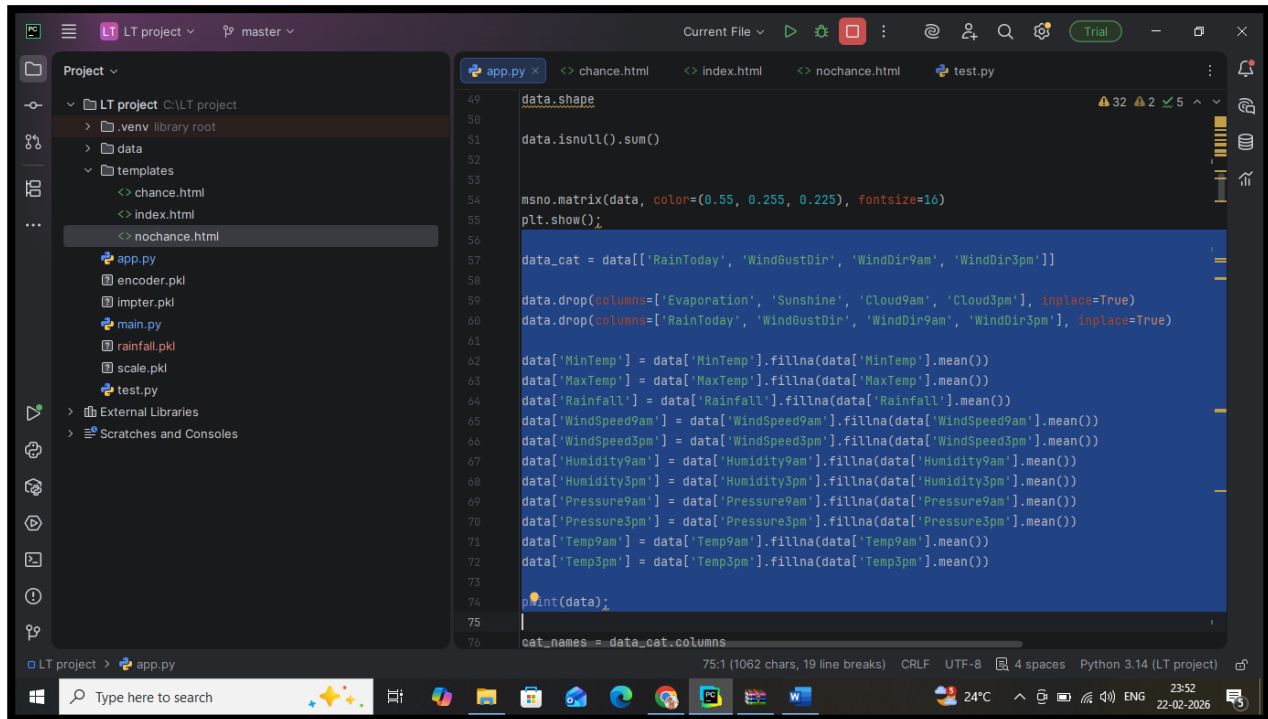
4. Imputing data using the Imputation method in sklearn.

Simple imputer

- Filling Nan values with mean, median, and mode using fillna () method.
- We are using the mean method to fill the missing data, where the data type of the columns is numeric,
- We are using the simple imputer method, which is in sklearn library from the module impute.

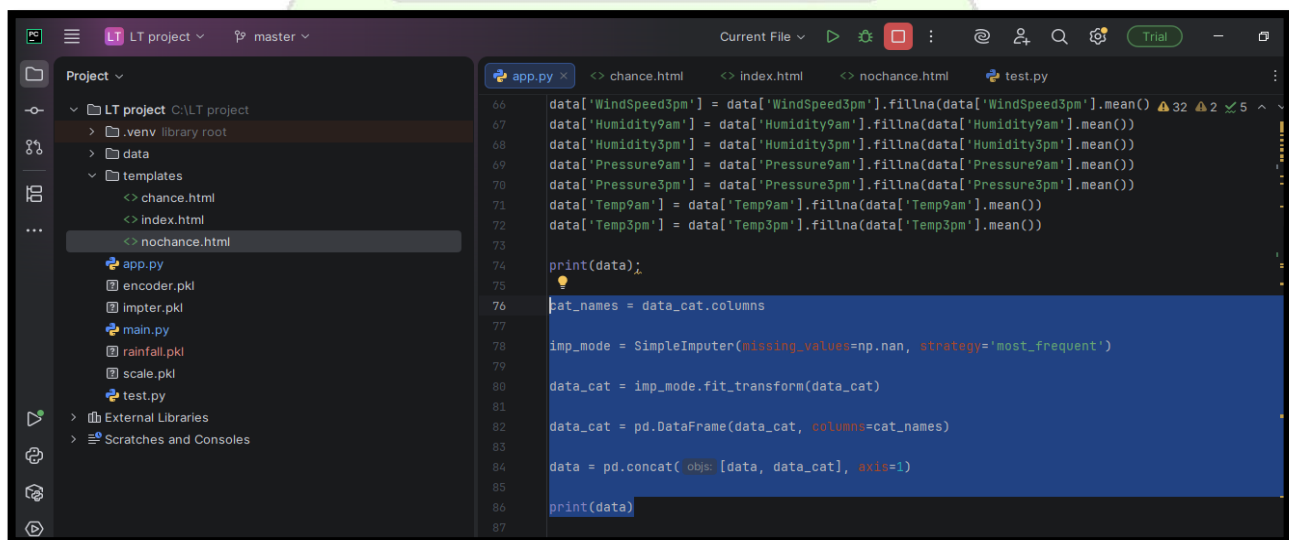
EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

- In impute method, for filling missing values of categorical data type we are using the most frequent method



```
49 data.shape
50 data.isnull().sum()
51
52
53 msno.matrix(data, color=(0.55, 0.255, 0.225), fontsize=16)
54 plt.show()
55
56 data_cat = data[['RainToday', 'WindGustDir', 'WindDir9am', 'WindDir3pm']]
57
58 data.drop(columns=['Evaporation', 'Sunshine', 'Cloud9am', 'Cloud3pm'], inplace=True)
59 data.drop(columns=['RainToday', 'WindGustDir', 'WindDir9am', 'WindDir3pm'], inplace=True)
60
61 data['MinTemp'] = data['MinTemp'].fillna(data['MinTemp'].mean())
62 data['MaxTemp'] = data['MaxTemp'].fillna(data['MaxTemp'].mean())
63 data['Rainfall'] = data['Rainfall'].fillna(data['Rainfall'].mean())
64 data['WindSpeed9am'] = data['WindSpeed9am'].fillna(data['WindSpeed9am'].mean())
65 data['WindSpeed3pm'] = data['WindSpeed3pm'].fillna(data['WindSpeed3pm'].mean())
66 data['Humidity9am'] = data['Humidity9am'].fillna(data['Humidity9am'].mean())
67 data['Humidity3pm'] = data['Humidity3pm'].fillna(data['Humidity3pm'].mean())
68 data['Pressure9am'] = data['Pressure9am'].fillna(data['Pressure9am'].mean())
69 data['Pressure3pm'] = data['Pressure3pm'].fillna(data['Pressure3pm'].mean())
70 data['Temp9am'] = data['Temp9am'].fillna(data['Temp9am'].mean())
71 data['Temp3pm'] = data['Temp3pm'].fillna(data['Temp3pm'].mean())
72
73 print(data)
74
75 cat_names = data_cat.columns
```

- From the heat map, we see that there are missing values in the dataset.



```
66 data['WindSpeed3pm'] = data['WindSpeed3pm'].fillna(data['WindSpeed3pm'].mean())
67 data['Humidity9am'] = data['Humidity9am'].fillna(data['Humidity9am'].mean())
68 data['Humidity3pm'] = data['Humidity3pm'].fillna(data['Humidity3pm'].mean())
69 data['Pressure9am'] = data['Pressure9am'].fillna(data['Pressure9am'].mean())
70 data['Pressure3pm'] = data['Pressure3pm'].fillna(data['Pressure3pm'].mean())
71 data['Temp9am'] = data['Temp9am'].fillna(data['Temp9am'].mean())
72 data['Temp3pm'] = data['Temp3pm'].fillna(data['Temp3pm'].mean())
73
74 print(data)
75
76 cat_names = data_cat.columns
77
78 imp_mode = SimpleImputer(missing_values=np.nan, strategy='most_frequent')
79
80 data_cat = imp_mode.fit_transform(data_cat)
81
82 data_cat = pd.DataFrame(data_cat, columns=cat_names)
83
84 data = pd.concat([data, data_cat], axis=1)
85
86 print(data)
87
```

DATA VISUALIZATION:

1. Correlation

- data.Corr() gives the correlation between the columns
- Correlation is a statistical term describing the degree to which two variables move in coordination with one another.

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

- If the two variables move in the same direction, then those variables are said to have a positive correlation.
- If they move in opposite directions, then they have a negative correlation.
- We have to use this code for Correlation

```
data.corr()
```

	Location	MinTemp	MaxTemp	Rainfall	WindGustSpeed	WindSpeed9am	WindSpeed3pm	Humidity9am	Humidity3pm	Pressure9am	Pressure3pm
Location	1.000000	-0.006195	-0.020490	-0.003456	0.069158	0.077025	0.064150	-0.002063	0.011040	0.036480	0.046338
MinTemp	-0.006195	1.000000	0.733919	0.103315	0.173361	0.174938	0.174187	-0.232382	0.005930	-0.424336	-0.424336
MaxTemp	-0.020490	0.733919	1.000000	-0.074204	0.066298	0.014580	0.050374	-0.499789	-0.499714	-0.309077	-0.309077
Rainfall	-0.003456	0.103315	-0.074204	1.000000	0.127263	0.085981	0.056766	0.221395	0.249617	-0.159677	-0.159677
WindGustSpeed	0.069158	0.173361	0.066298	0.127263	1.000000	0.577790	0.658334	-0.209226	-0.025751	-0.426586	-0.426586
WindSpeed9am	0.077025	0.174938	0.014580	0.085981	0.577790	1.000000	0.513067	-0.269031	-0.031001	-0.215159	-0.215159
WindSpeed3pm	0.064150	0.174187	0.050374	0.056766	0.658334	0.513067	1.000000	-0.144264	0.015782	-0.277469	-0.277469
Humidity9am	-0.002063	-0.232382	-0.499789	0.221395	-0.209226	-0.269031	-0.144264	1.000000	0.659865	0.131593	0.131593
Humidity3pm	0.011040	0.005930	-0.499714	0.249617	-0.025751	-0.031001	0.015782	0.659865	1.000000	-0.025781	-0.025781
Pressure9am	0.036480	-0.424336	-0.309077	-0.159677	-0.426586	-0.215159	-0.277469	0.131593	-0.025781	1.000000	0.959878
Pressure3pm	0.046338	-0.434021	-0.397412	-0.120366	-0.384622	-0.165038	-0.239725	0.176164	0.048554	0.959878	1.000000
Temp9am	-0.015597	0.897998	0.880085	0.011385	0.146761	0.128769	0.162143	-0.471141	-0.217573	-0.397746	-0.397746
Temp3pm	-0.022712	0.699824	0.969733	-0.077555	0.032245	0.005023	0.028444	-0.492441	-0.555778	-0.266291	-0.266291
RainToday	-0.004911	0.055644	-0.226474	0.500279	0.148264	0.100563	0.078361	0.348843	0.370561	-0.179226	-0.179226
WindGustDir	-0.005055	-0.136317	-0.212210	0.044855	0.137914	0.009762	0.084012	0.067956	0.063964	-0.120630	-0.120630
WindDir9am	-0.004434	-0.029638	-0.212589	0.085140	0.074638	0.109916	0.111072	0.088842	0.148893	-0.050314	-0.050314
WindDir3pm	0.008325	-0.158958	-0.181344	0.047905	0.136674	0.050200	0.089824	0.026077	-0.007182	-0.133633	-0.133633

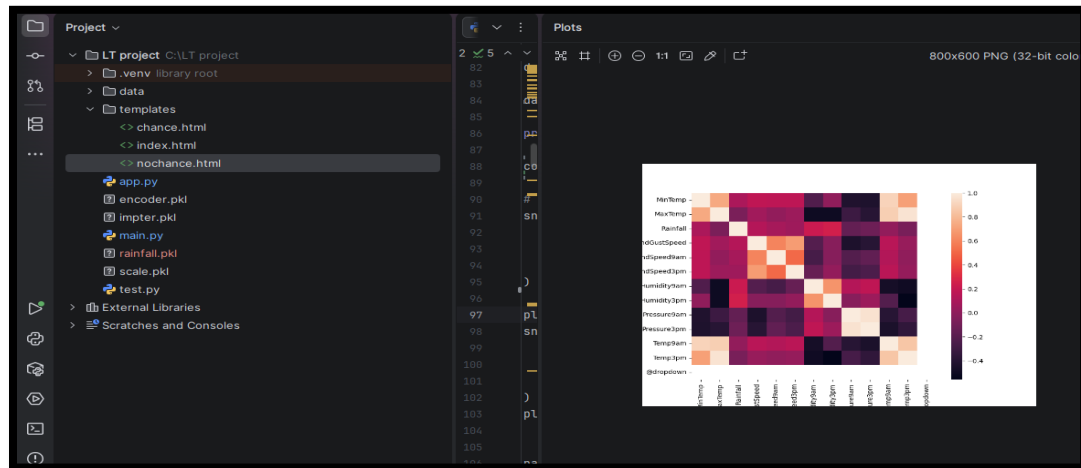
2.Heat-map:

- A heatmap is a graphical representation of data that uses a system of color-coding to represent different values. It is used to identify the correlation between the columns using a visual manner.

```
app.py x  chance.html  index.html  nochange.html  test.py
81
82 data_cat = pd.DataFrame(data_cat, columns=cat_names)
83
84 data = pd.concat([data, data_cat], axis=1)
85
86 print(data)
87
88 cor = data.corr(numeric_only=True)
89
90 # heatmap (same style as image)
91 sns.heatmap(
92     data=cor,
93     xticklabels=cor.columns.values,
94     yticklabels=cor.columns.values
95 )
96
```


EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

Output:

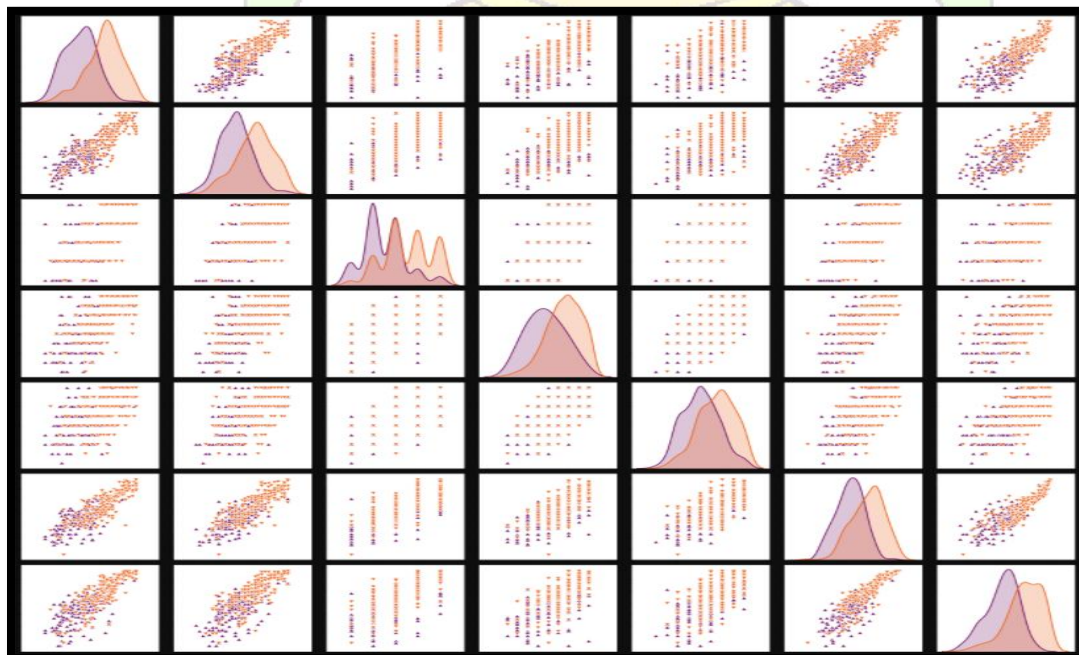


2.Pair Plot:

- Plot pairwise relationships in a dataset.

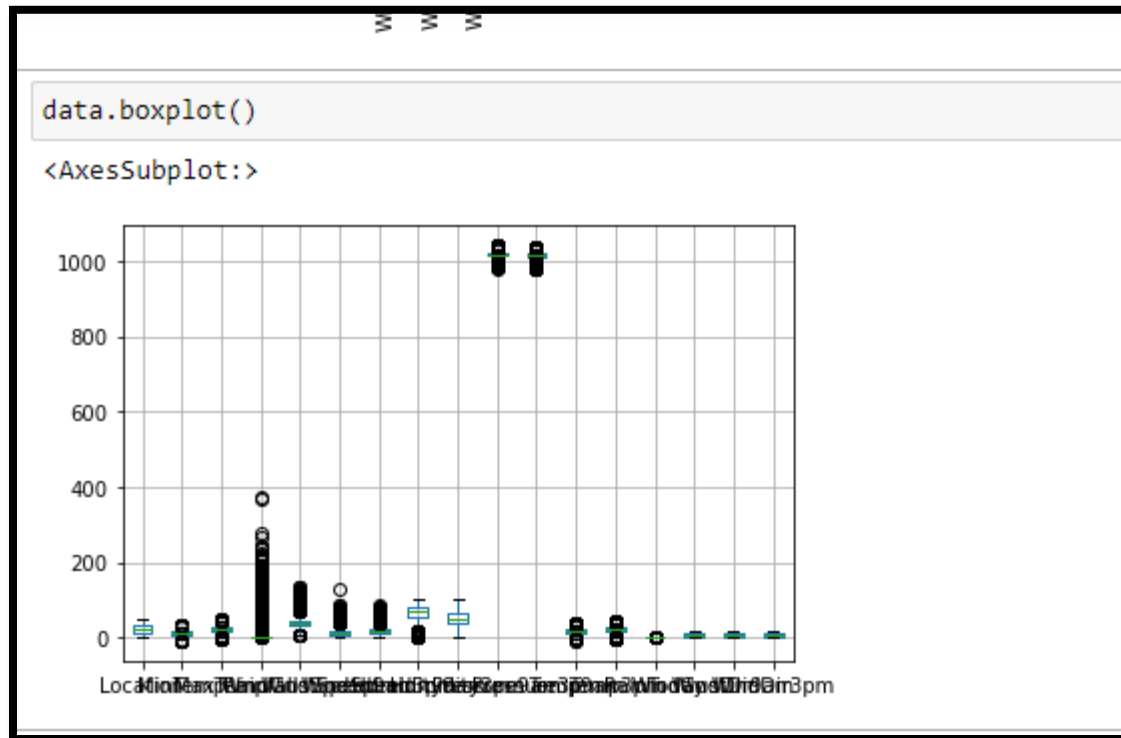
```
plt.figure(figsize=(8, 6))
sns.heatmap(
    cor,
    xticklabels=cor.columns.values,
    yticklabels=cor.columns.values
)
plt.show()
```

The output is as shown below



EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

3. Box Plot:



4. Joint plot:

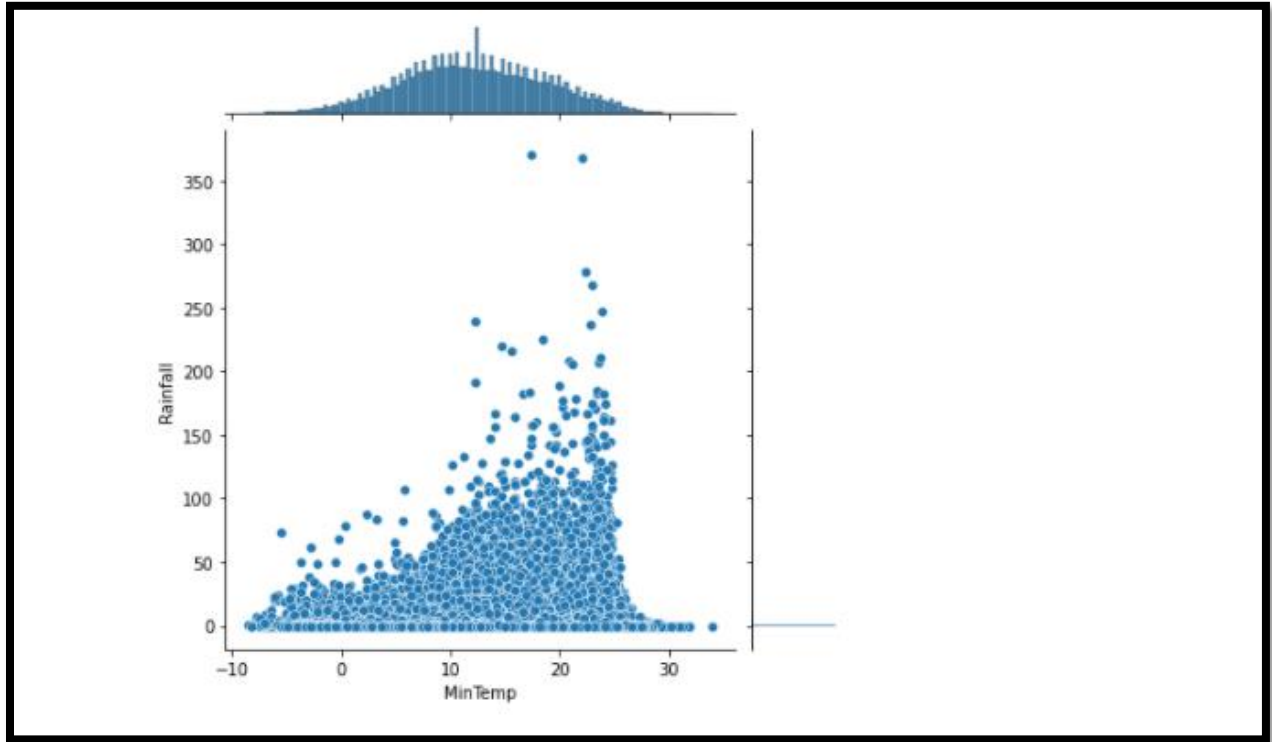
- We have to paste code.

The screenshot displays the PyCharm IDE interface. On the left, the Project Explorer shows a project named 'LT project' with a file structure including 'venv', 'data', and 'templates'. The 'templates' folder contains files 'chance.html', 'index.html', and 'nochance.html'. The 'app.py' file is selected, showing its contents in the main editor. The script imports 'matplotlib.pyplot' as 'plt' and 'pandas' as 'pd'. It reads 'data.csv' into a DataFrame 'data'. A boxplot is generated using 'plt.boxplot()', and a scatter plot is added as a subplot using 'fig.add_subplot()'. The scatter plot is titled 'MinTemp vs Rainfall' and uses 'alpha=0.6' for transparency. A histogram is also added as a subplot using 'fig.add_subplot()' and titled 'MinTemp vs Rainfall'. The plot is displayed using 'plt.show()'. The bottom status bar indicates the file encoding is UTF-8, the workspace has 4 spaces, and the Python version is 3.14 (LT project).

```
123 hue='RainToday',
124 diag_kind='kde',
125 palette=['purple', 'orange'],
126 markers=['^', 'v'],
127 plot_kws={'alpha': 0.6, 's': 25},
128 diag_kws={'fill': True}
129 )
130
131 plt.show()
132
133 data.boxplot(),
134 plt.show(),
135
136 x = data['MinTemp']
137 y = data['Rainfall']
138
139 fig = plt.figure(figsize=(8, 8))
140 grid = plt.GridSpec(nrows=4, ncols=4, hspace=0.3, wspace=0.3)
141
142 ax_scatter = fig.add_subplot(grid[1:4, 0:3])
143 ax_scatter.scatter(x, y, alpha=0.6)
144 ax_scatter.set_xlabel("MinTemp")
145 ax_scatter.set_ylabel("Rainfall")
146 ax_scatter.set_title("MinTemp vs Rainfall")
147
148 ax_histx = fig.add_subplot(grid[0, 0:3], sharex=ax_scatter)
149 ax_histx.hist(x, bins=30)
150 ax_histx.tick_params(axis='x', labelbottom=False)
```

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

Output:



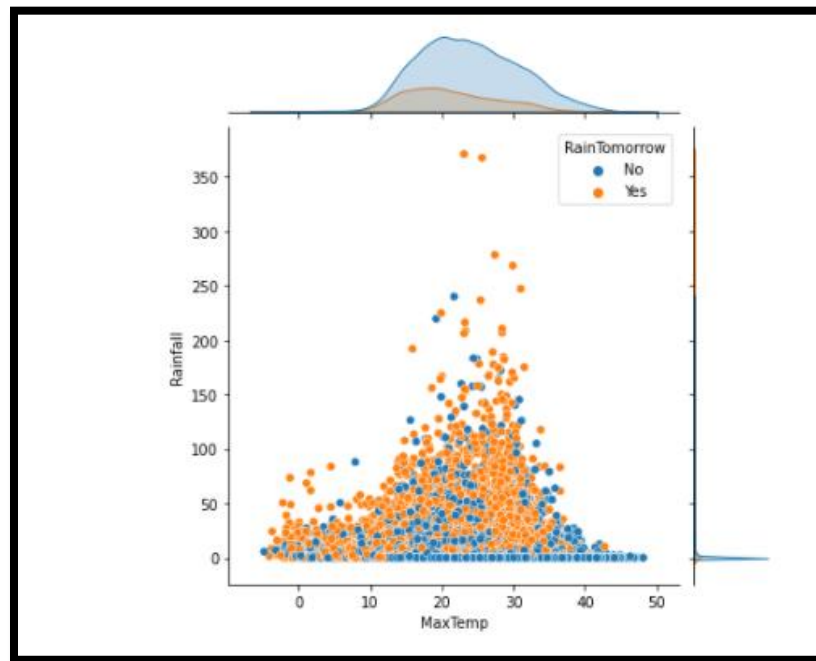
Join plot with hue function: తమసోపానా జ్యోతిర్గమయ

- We have to paste code.

```
134 plt.show()
135
136 x = data["MinTemp"]
137 y = data["Rainfall"]
138
139 fig = plt.figure(figsize=(8, 8))
140 grid = plt.GridSpec(nrows=4, ncols=4, hspace=0.3, wspace=0.3)
141
142 ax_scatter = fig.add_subplot(grid[1:4, 0:3])
143 ax_scatter.scatter(x, y, alpha=0.6)
144 ax_scatter.set_xlabel("MinTemp")
145 ax_scatter.set_ylabel("Rainfall")
146 ax_scatter.set_title("MinTemp vs Rainfall")
147
148 ax_histx = fig.add_subplot(grid[0, 0:3], sharex=ax_scatter)
149 ax_histx.hist(x, bins=30)
150 ax_histx.tick_params(axis="x", labelbottom=False)
151 ax_histx.set_ylabel("Frequency")
152
153 ax_histy = fig.add_subplot(grid[1:4, 3], sharey=ax_scatter)
154 ax_histy.hist(y, bins=30, orientation="horizontal")
155 ax_histy.tick_params(axis="y", labelleft=False)
156 ax_histy.set_xlabel("Frequency")
157 plt.show()
158
159 data = data[data["MinTemp"] > 10]
```

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

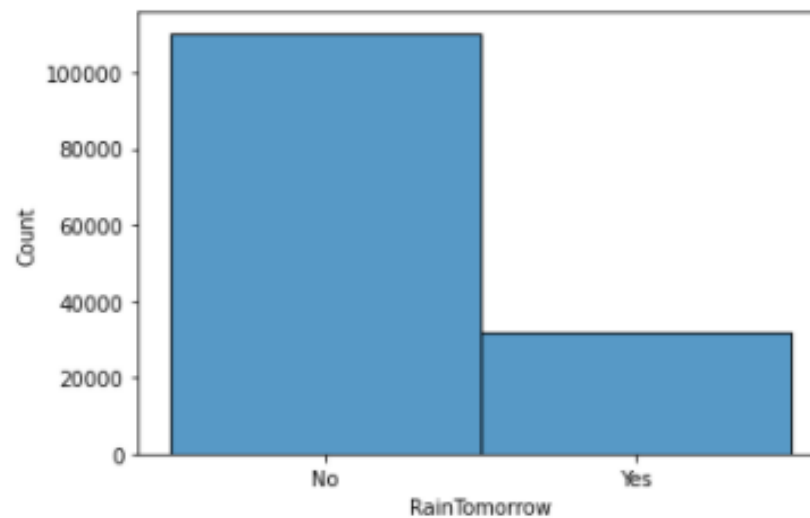
Output:



5. Hist plot:

```
In [18]: sns.histplot(data['RainTomorrow'])
```

```
Out[18]: <AxesSubplot:xlabel='RainTomorrow', ylabel='Count'>
```



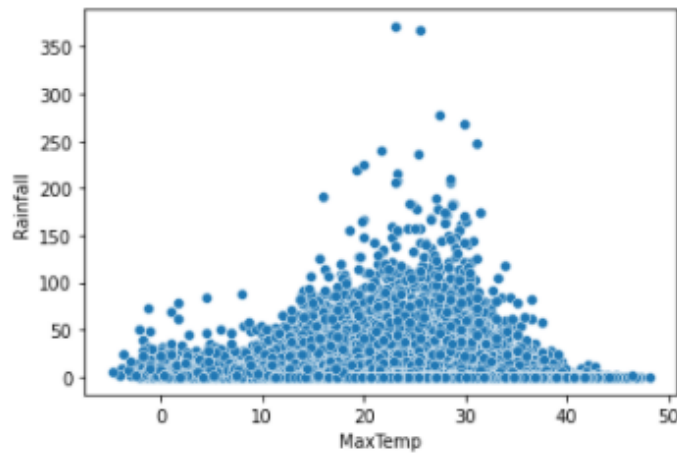
EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

6. Scatter plot:

```
In [20]: sns.scatterplot(data['MaxTemp'],data['Rainfall'])
```

C:\Users\SmartbridgePC\anaconda4\lib\site-packages\seaborn_decorators.py:36: FutureWarning: positional arguments: x, y. From version 0.12, the only valid positional argument will be `data`. Explicit keyword arguments will result in an error or misinterpretation.
warnings.warn(

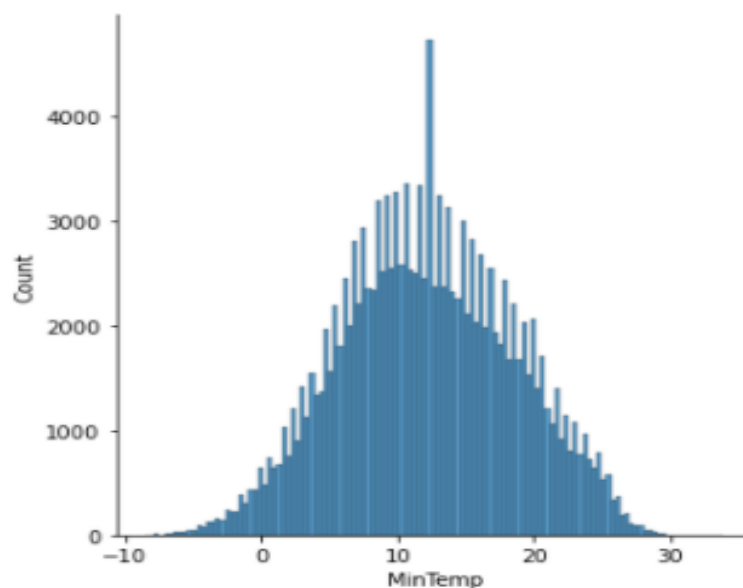
```
Out[20]: <AxesSubplot:xlabel='MaxTemp', ylabel='Rainfall'>
```



7. A Distplot or distribution plot:

```
In [19]: sns.displot(data['MinTemp'])
```

```
Out[19]: <seaborn.axisgrid.FacetGrid at 0x20b468ddfa0>
```



EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

SPLITTING THE DATASET INTO DEPENDENT AND INDEPENDENT VARIABLE

- We have to paste the Below code.
- Splitting the Datasets into Dependent and Independent variable.

```
#splitting x and y values  
y = data['RainTomorrow']  
x = data.drop('RainTomorrow',axis=1)
```

Feature Scaling:

- There is huge disparity between the x values so let us use feature scaling.
- Feature scaling is a method used to normalize the range of independent variables or features of data.
- We have to paste the below code.

standardizing the data

```
: from sklearn.preprocessing import StandardScaler  
  
: #splitting x and y values  
y = data['RainTomorrow']  
x = data.drop('RainTomorrow',axis=1)  
  
: names = x.columns # Loading the names of the x_features  
  
: names  
Index(['Location', 'MinTemp', 'MaxTemp', 'Rainfall', 'WindGustSpeed',  
      'WindSpeed9am', 'WindSpeed3pm', 'Humidity9am', 'Humidity3pm',  
      'Pressure9am', 'Pressure3pm', 'Temp9am', 'Temp3pm', 'RainToday',  
      'WindGustDir', 'WindDir9am', 'WindDir3pm', 'year', 'month', 'day'],  
      dtype='object')  
  
: sc = StandardScaler() # initializing the standardscaler  
  
: x = sc.fit_transform(x) # fitting an transforming the data into standard  
  
: x = pd.DataFrame(x,columns=names) #scaled data turns into array format,converting them into dataframe
```

EXPLORATORY ANALYSIS OF RAIN FALL DATA IN INDIA FOR AGRICULTURE

Splitting the data into Train and Test:

- Train Dataset: Used to fit the machine learning model.
- Test Dataset: Used to evaluate the fit machine learning model.
- Now split our dataset into train set and test using train_test_split class from scikit learn library.

```
from sklearn import model_selection
```

```
x_train,x_test,y_train,y_test = model_selection.train_test_split(x,y,test_size =0.2,random_state =0)
```

